MILESTONE 2 REPORT

Classification Phase

Team ID: CS_28 **Target Variable:** JobSatLevel (Low / Average / High) **Dataset:** Stack Overflow Annual Developer Survey

#	Name	ID
1	علي عبد الهادي راتب عبد الهادي	2023170370
2	حسام أحمد فتحي النمر	2023170190
3	رحاب صبحي عبد الواحد عبد الباقي	2023170216
4	روان أحمد محمد غريب	2023170222
5	إبراهيم أحمد شكري عبد الفتاح الطوخي	2023170009
6	أحمد أيمن محمود خليفة عبد الله	2023170026

Performance Summary

This section summarizes the three required comparison dimensions: classification accuracy, total training time, and total test/prediction time. Seven classifiers were compared on the same processed train/test split.

Model	Accuracy	Balanced Accuracy	Macro F1	Training Time (s)	Test Time (s)
XGBoost	0.5312	0.4541	0.4521	8.9795	0.0329
Random Forest with CV	0.5260	0.4371	0.4222	4.4136	0.3894
Logistic Regression	0.5212	0.4454	0.4412	6.7829	0.0057
Decision Tree with CV	0.5191	0.4329	0.4245	0.3309	0.0076
Linear SVM	0.5064	0.4945	0.4942	3.5675	0.0293
Random Forest	0.4765	0.4974	0.4782	3.4818	0.3748
Decision Tree	0.4515	0.4736	0.4536	1.0405	0.0150

The best overall accuracy was achieved by XGBoost (0.5312), followed closely by the CV-tuned Random Forest (0.5260) and Logistic Regression (0.5212). While the accuracy values are close, the timing results show clear differences in computational cost.

Figure 1. Model Accuracy Comparison

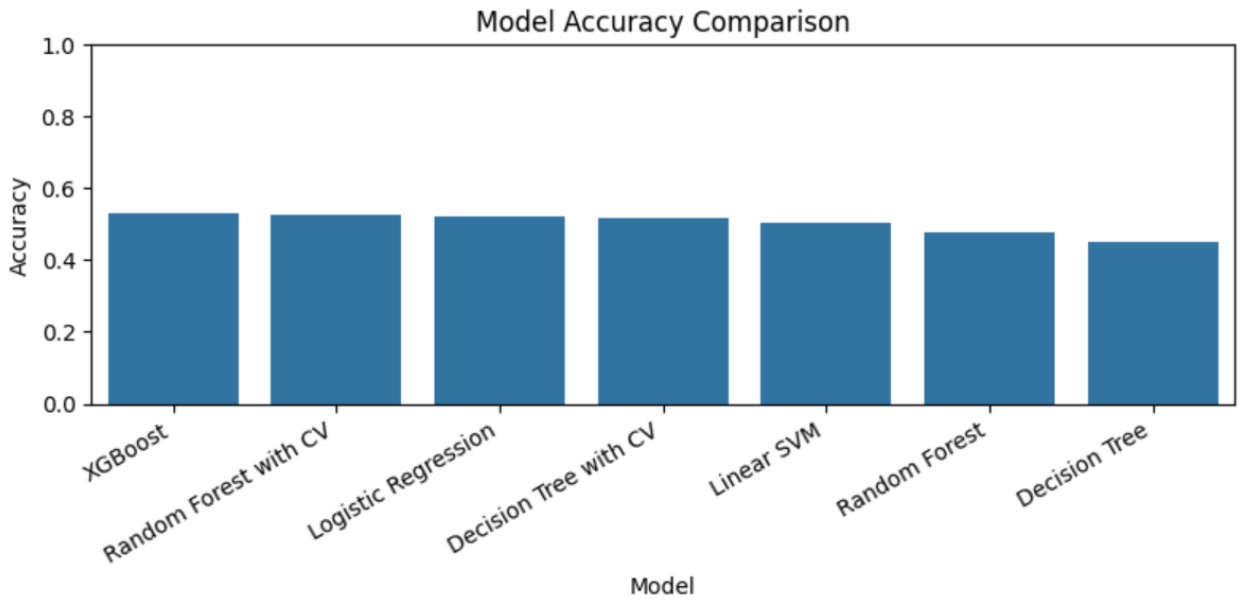


Figure 2. Model Training Time Comparison

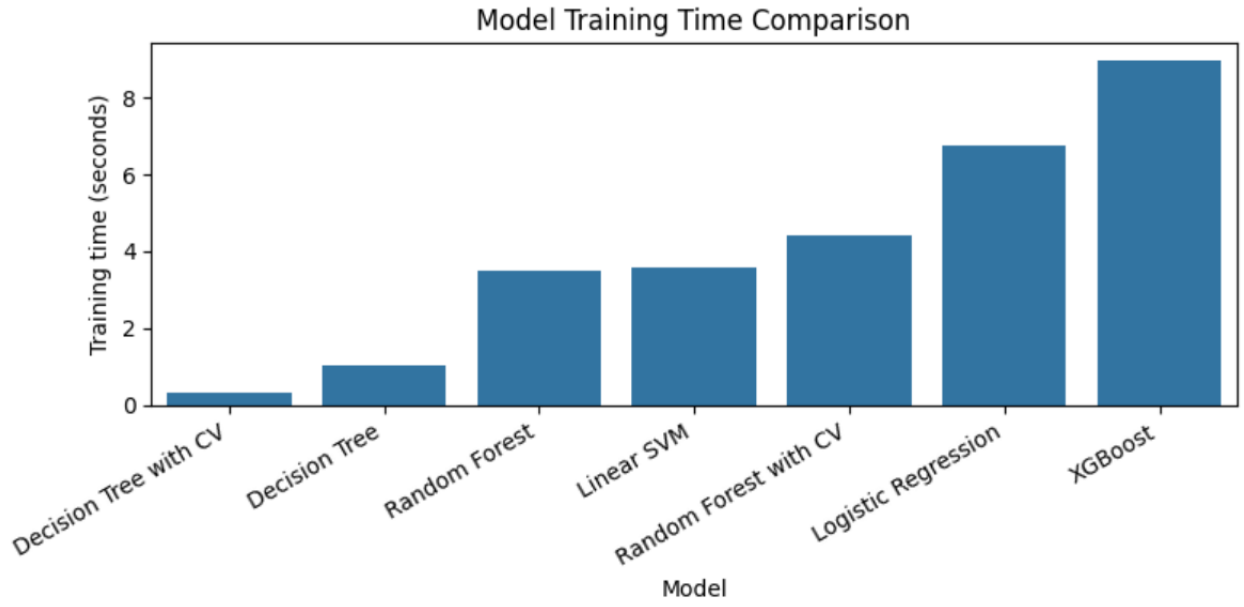
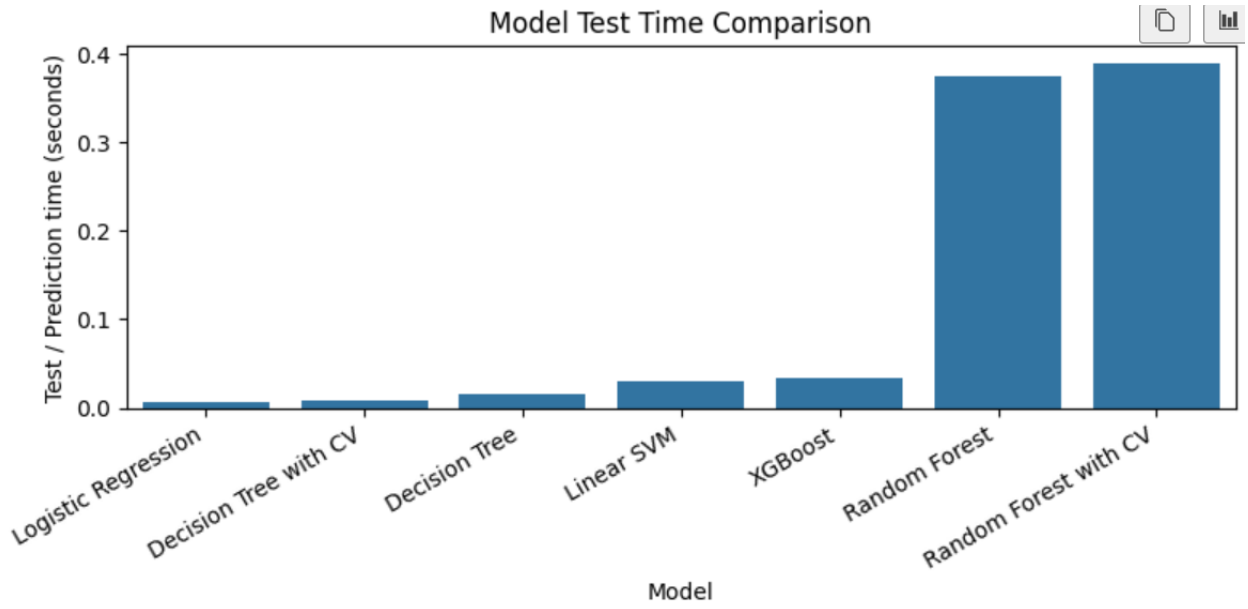


Figure 3. Model Test / Prediction Time Comparison



Key takeaway: ensemble models, especially XGBoost and Random Forest with CV, offered the strongest accuracy, but they also required more training or prediction time than simpler models. Test time was still acceptable for all models.

Feature Selection Process

The feature selection process in this classification phase differed from the previous regression phase because the target was converted into three ordinal classes: Low, Average, and High. Therefore, classification-oriented methods were more suitable than purely correlation-based regression methods.

Instead of relying mainly on regression correlations, the pipeline used classification-focused feature selection and feature engineering techniques. These included Mutual Information for non-linear relationships, chi-square analysis and Cramér's V for categorical associations, Kruskal-Wallis tests for ordinal/numeric features, multi-hot encoding for multi-choice survey answers, ordinal/one-hot encoding for categorical columns, and StandardScaler for the final training matrix.

The features were organized into thematic blocks: Main AI Usage, AI Agent & Developer Profile, Compensation & Technology Attitude, Stack Overflow Community, Work/Database, and Tools & Platforms. Each block was preprocessed separately, then combined into the final feature matrix used by the classification models.

Multiple-choice question engineering:

Two preprocessing methods were used for columns where respondents could select more than one answer. The first method split each response by the semicolon character (;), cleaned the separated items, and then applied multi-hot encoding. This transformed every possible selected option into its own binary feature, where 1 means the respondent selected that option and 0 means they did not.

The second method converted the same type of multiple-choice response into a count feature by calculating the number of semicolons plus one. For example, if a respondent selected three tools, the text would contain two semicolons, so the engineered feature value becomes 2 + 1 = 3. This count-based feature captures how many options the respondent selected without expanding the column into many binary variables.

Feature Block	Selection / Engineering Method
Main AI Usage	Top features selected using Mutual Information; WorkExp kept as a domain-driven feature; multi-choice columns were multi-hot encoded.
AI Agent & Developer Profile	AI challenge responses were converted into agreement scores; top features included DevType_encoded, OrgSize, ICorPM, AI_Challenges_Total, and AIAgent_Uses_Count.
Compensation & Survey	CompTotal was converted to USD; survey attitude items were retained; PCA analysis showed high dimensionality, so the information was preserved for tree-based models.
Stack Overflow Community	SOComm and SOPartFreq were important; low-information friction/action features were reduced; tag columns were selected using MI-based methods.
Tools & Platforms	Technology and platform columns were multi-hot encoded; RemoteWork and Employment were encoded into model-ready numerical features.

The initial feature-selection intuition was partially proved. AI-related attitudes, developer profile features, and work experience showed useful signal. However, compensation alone was less predictive than expected, which suggests that job satisfaction is affected by several non-linear and human-centered factors rather than salary only.

Hyperparameter Tuning

The required tuning experiment focused on two hyperparameters, and each hyperparameter was tested using exactly three different values. The two tuned parameters were Linear SVM C and Random Forest n_estimators.

Model	Tuned Parameter	Value	Accuracy	Balanced Accuracy	Macro F1	Training Time (s)
Linear SVM	C	0.1	0.5064	0.4945	0.4942	2.2156
Linear SVM	C	1.0	0.5064	0.4945	0.4942	2.1154
Linear SVM	C	10.0	0.5064	0.4945	0.4942	2.2374
Random Forest	n_estimators	100	0.4688	0.4905	0.4708	1.7511
Random Forest	n_estimators	200	0.4765	0.4974	0.4782	3.0983
Random Forest	n_estimators	300	0.4761	0.4961	0.4775	5.5128

For Linear SVM, changing C between 0.1, 1.0, and 10.0 did not change accuracy, balanced accuracy, or macro F1. This indicates that the SVM performance was stable and not very sensitive to this regularization range. The value C=1.0 is still a reasonable choice because it achieved the same performance with the lowest training time in this experiment.

For Random Forest, increasing n_estimators from 100 to 200 improved accuracy from 0.4688 to 0.4765 and improved balanced accuracy and macro F1. Increasing the number of trees further to 300 did not improve performance and only increased training time. Therefore, n_estimators=200 was the best practical value among the three tested options.

Overall, hyperparameter tuning improved the understanding of the models even when the numerical gain was moderate. It showed that more complex settings are not automatically better; after a certain point, they can increase training time without improving prediction quality.

Conclusion and Project Intuition

This milestone implemented a complete classification pipeline for predicting developer job satisfaction levels (Low, Average, High). The pipeline included preprocessing, feature engineering, feature selection, model training, hyperparameter tuning, and performance comparison.

The final results support the intuition that job satisfaction is a non-linear classification problem. XGBoost achieved the highest accuracy, while Random Forest with CV was very close. Linear SVM showed the best balanced accuracy and macro F1 among the top models, which means it handled class balance relatively well.

The project intuition was partially proved. AI tool attitudes, developer role/profile, work experience, and community participation contributed useful information. However, the results also disproved the idea that compensation alone would dominate job satisfaction prediction. The moderate accuracy values show that job satisfaction is complex and cannot be fully explained by the available structured survey features.

Future improvements could include a wider hyperparameter grid, more detailed class-imbalance treatment, feature interaction analysis, stacking/ensemble blending, and text-based features if free-text survey responses are available.

Best Performing Model	XGBoost — Accuracy: 0.5312 | Balanced Accuracy: 0.4541 | Macro F1: 0.4521